Объект FormData, часть 2

- Работа с полем type=’file’
- Это важно понимать
- Метод append для добавления файла
- Практика работы с формами
- Пример асинхронного обмена данными

Работа с полем type=’file’

Для того чтобы можно было загружать на сервер один или несколько файлов, в форме применяется специальный элемент. Как правило в браузерах такой элемент отображается как **текстовое поле**, рядом с которым располагается **кнопка** с надписью **Обзор**, в некоторых браузерах кнопка может называться **Выберите файл** (Browse / Choose File и пр).

Внешний вид элемента выбора зависит от браузера.

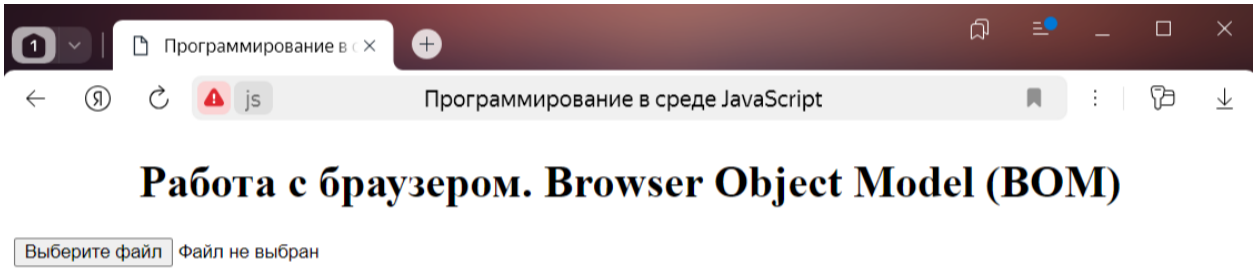


Рис. 1. Браузер Yandex

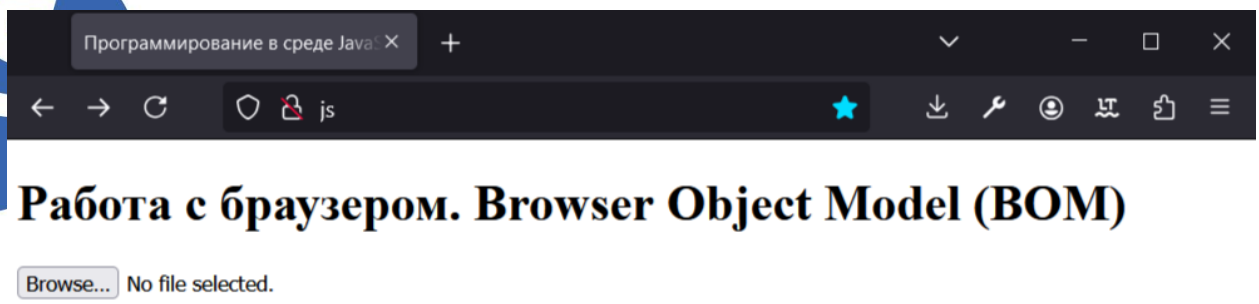


Рис. 2. Браузер Firefox

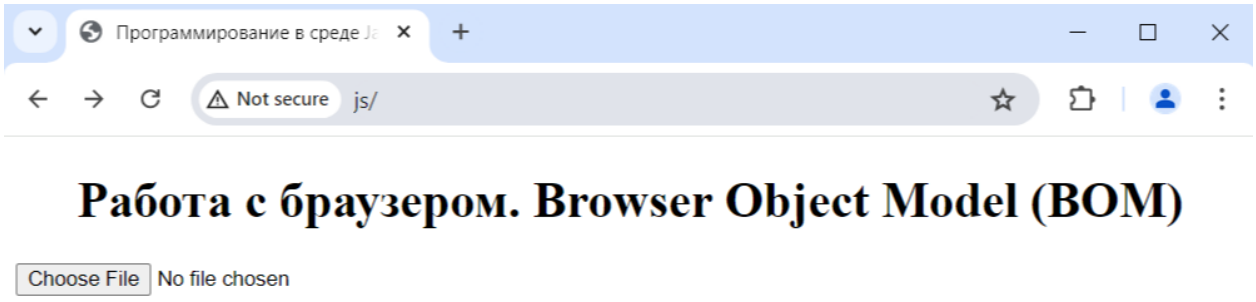


Рис. 3. Браузер Google Chrome

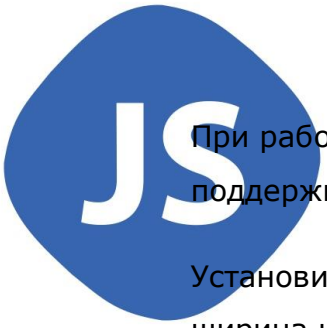
При нажатии на кнопку открывается окно для выбора файла, где можно указать, какой файл будет загружен на сервер.

Синтаксис элемента отправки файла, следующий:

```
<input type="file" attribute>
```

Возможные атрибуты (attribute) поля:

- **accept** – устанавливает фильтр на типы файлов, которые пользователь может отправить через поле загрузки файлов;
- **size** – ширина текстового поля, которое определяется числом символов моноширинного шрифта;
- **multiple** – позволяет выбирать и загружать сразу **несколько файлов**;
- **name** – имя поля, используется для его идентификации обработчиком формы.



При работе с элементом выбора файла стоит учитывать, не все браузеры поддерживают новые атрибуты.

Установить ширину поля через атрибут **size** можно, но в действительности ширина никак не влияет на внешний вид элемента выбора. На браузеры Safari и Google Chrome этот атрибут вообще никакого воздействия не оказывает.

Атрибут **multiple** более важен, он позволяет не ограничиваться одним файлом для выбора, а указать их сразу **несколько для одновременной загрузки**.

Если атрибут **accept** не указывать, тогда добавляются и загружаются файлы любого типа. Наличие **accept** позволяет ограничить выбор файла, что особенно важно, когда требуется загрузить только изображение или видео. В качестве значения выступает MIME-тип, несколько значений разделяются между собой запятой.

В таблице показаны некоторые допустимые значения атрибута **accept**.

Значение	Описание
image/jpeg	Только файлы в формате JPEG
image/jpeg,image/png	Только файлы в формате JPEG и PNG
image/*	Любые графические файлы
image/*,video/*	Любые графические и видеофайлы

Форма с атрибутом **accept** может иметь следующий вид:

```
<form id="frm">
    <!-- -->
    Изображение: <input type="file" name="picture"
    accept="image/*">
    <input type="submit">
</form>
```

Safari, Firefox, IE не поддерживают **accept**. IE полностью игнорирует **multiple**.

Обратите внимание, атрибут **multiple** несколько изменяет внешний вид поля.

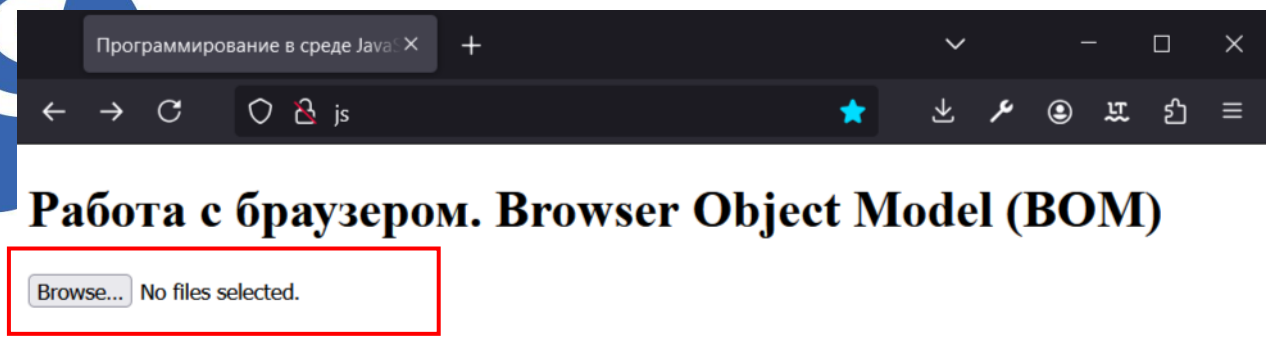


Рис. 4. Кнопка мультизагрузки

> ***Важно***. При **синхронной** отправке файлов серверному сценарию форма должна иметь следующие атрибуты:
>
> - метод отправки данных: **method="post"**
> - атрибут enctype: **enctype="multipart/form-data"**

В текущем уроке это условие не принципиально. Выбранный файл не отправляется, учимся обрабатывать данные формы на клиентской стороне.

И тем не менее, объекты **FormData** всегда отсылаются с заголовком **Content-Type: multipart/form-data**, именно этот способ кодировки позволяет отсылать файлы.

> ***Важно***. При **асинхронной** отправке файлов серверному сценарию поля <input type="file"> будут отправляться, как это происходит в случае обычной формы. Объект **FormData** позаботится обо всем сам.

example_1.

```
<form id="frm">
    <!-- -->
    <input type="file" name="file">
</form>
```

```
<script>
    // создаем объект FormData
    let form = document.querySelector("#frm");
    let data = new FormData (form);
    // получаем ссылку на элемент type="file"
    let file = data.get("file");
    // тестируем вывод
    console.log(file); // File
    console.log(file.name); // <empty string>
</script>
```

В демонстрационном примере **example_2** сравним **значение** элемента выбора **до** выбора пользователем файла, и **после**. Это не мелочи, все будет играть свою роль при дальнейшей работе.

example_2.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file">
</form>
<script>
    // получаем ссылку на элемент type="file"
    let form = document.querySelector("#frm");
    let data = new FormData (form);
    let file = data.get("file");
    // начальные данные объекта FormData
    console.log(file); // File
    console.log(file.name); // <empty string>
```

```
    console.log('----------');
    // обработка события - выбор файла
    document.getElementById('file').onchange = function() {
        // используя ссылку на форму создадим новый объект
        FormData
        let data = new FormData (form);
        let file = data.get("file");
        // данные объекта FormData после выбора файла
        console.log(file);  // File
        console.log(file.name); // ...
    }
</script>
```

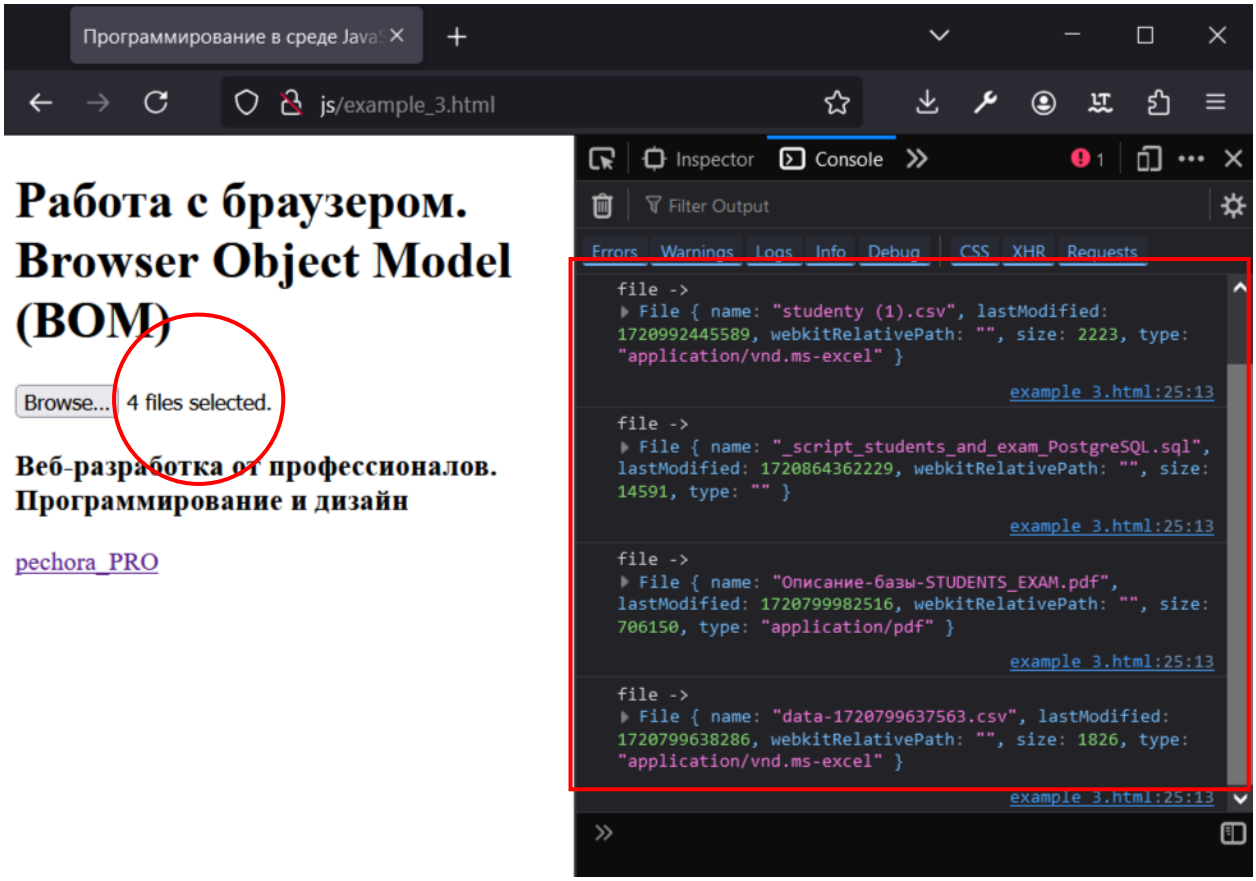
Используем свойство **multiple** для **множественного** выбора файла.

Множественный выбор возможен при выборе файлов с нажатой кнопкой **Ctrl**.

example_3.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file" multiple="multiple">
</form>
<script>
    // обработка события - выбор файла
    document.getElementById('file').onchange = function() {
        // получаем ссылку на элемент type="file"
        let form = document.querySelector("#frm");
        let data = new FormData (form);
        for (let [key, file] of data){
```

```
            console.log(key, '->', file)
        }
    }
</script>
```



В реальных приложениях в качестве имени элемента выбора файла принято использовать обозначение **массива**:

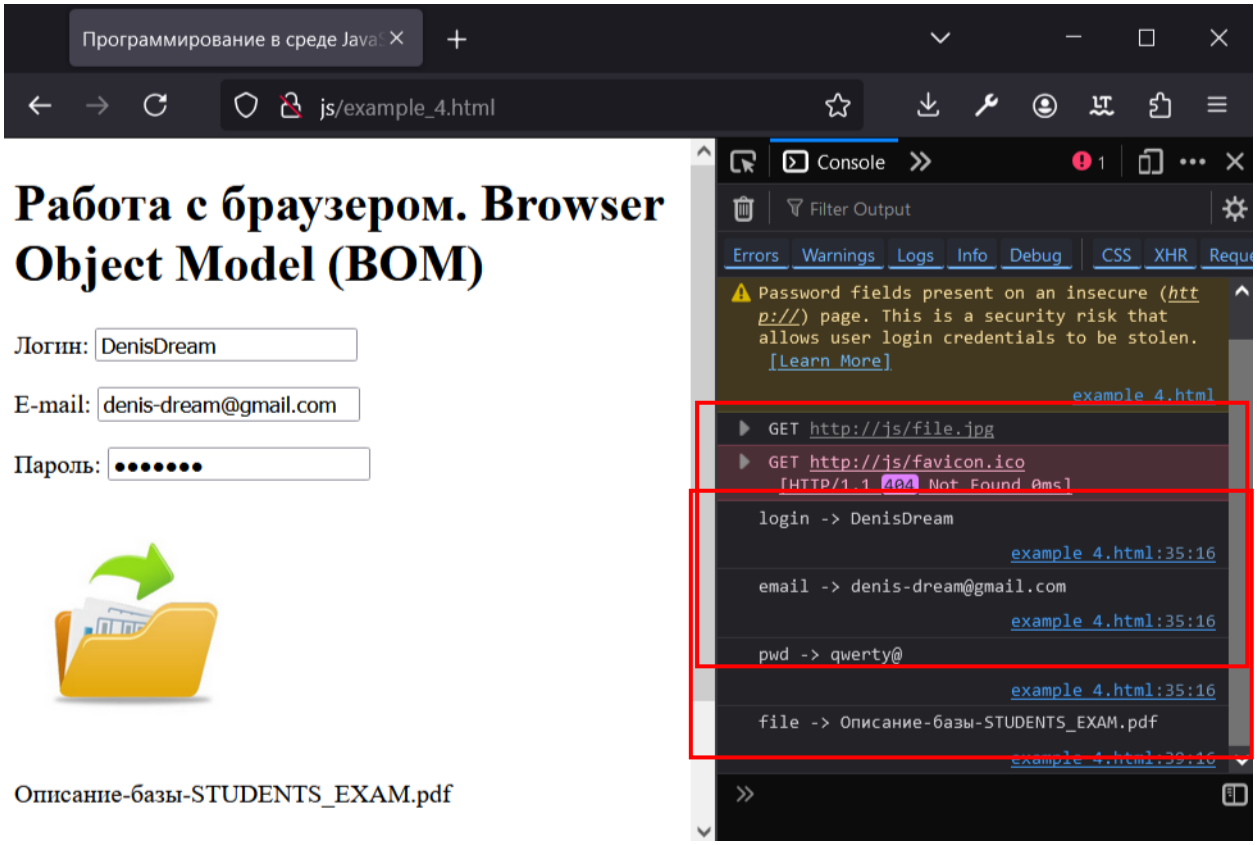
```
// массив в качестве имени элемента
<input id="file" type="file" name="file[]" multiple="multiple">
```

Эта техника позволяет автоматизировать мультифайловую обработку данных на сервере, структура данных является **более благоприятной для цикла** (итерируемой).

Примечание. Все это и многое другое в курсе: **Программирование на**

языке PHP. Путь к Middle-разработчику.

В демонстрационном примере **example_4** спрячем кнопку выбора файла за **пользовательским** изображением.



example_4.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <label for="file"><img src="file.jpg" width="150px"></label>
    <input id="file" type="file" name="file" hidden>
</form>
<span id="fileName">Файл не выбран</span>
```

```
<script>
    // обрабатываем событие выбор файла
    document.getElementById('file').onchange = function() {
    // создаем объект FormData
    data = new FormData (document.getElementById('frm'));
    // перебираем параметры объекта
    for (let [key, val] of data.entries()) {
        // если значение параметра - строка
        if(typeof val === 'string') {
            console.log(key + ' -> ' + val);
        }
        // если значение параметра - объект File
        else
            console.log(key + ' -> ' + val.name);
        }
        // дабавляем название выбранного файла
        let span = document.getElementById('fileName');
        span.innerText = data.get("file").name;
    };
</script>
```

Это важно понимать

До сих пор мы получали данные о загружаемом файле через объект **FormData**, но важно понимать следующее:

- к любому элементу формы можно обратиться как к **HTML–элементу** дерева DOM;



- к любому элементу формы можно обратиться как к свойству специального объекта **FormData**;
- свойства элемента будут зависеть от **способа его получения**.

Разбираем подробно другие способы получения доступа к элементу выбора файла.

Доступ к элементу через this

example_5.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file">
</form>
<script>
    // получаем ссылку на элемент type="file"
    let form = document.querySelector("#frm");
    let data = new FormData (form);
    // обработка события - выбор файла
    document.getElementById('file').onchange = function() {
        // доступ к элементу через this
        console.log(this); // input
        console.log(this.files); // FileList
        console.log(this.files[0]); // File
    }
</script>
```

Примечание. При работе с **this** помните, **this** коварен, и зависит от



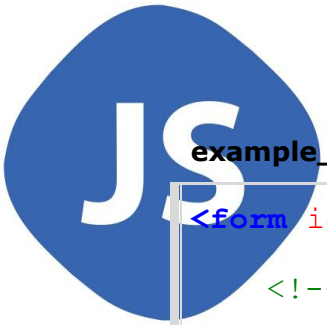
контекста его использования.

Доступ к элементу через event

example_6.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file">
</form>
<script>
    // получаем ссылку на элемент type="file"
    let form = document.querySelector("#frm");
    let data = new FormData (form);
    // обработка события - выбор файла
    document.getElementById('file').onchange = function(e) {
        // доступ к элементу через event
        let input = e.target;
        console.log(input); // input
        console.log(input.files); // FileList
        console.log(input.files[0]); // File
        console.log(input.files[0].name); // ...
    }
</script>
```

Доступ к элементу через getElementById



example_7.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file">
</form>
<script>
    // получаем ссылку на элемент type="file"
    let form = document.querySelector("#frm");
    let data = new FormData (form);
    // обработка события - выбор файла
    document.getElementById('file').onchange = function(e) {
        // доступ к элементу через getElementById
        let input = document.getElementById('file');
        console.log(input); // input
        console.log(input.files); // FileList
        console.log(input.files[0]); // File
        console.log(input.files[0].name); // ...
    }
</script>
```

Подводим итог

example_8.

```
<form id="frm">
    <!-- -->
    <input id="file" type="file" name="file">
```

```
</form>
<script>
    // получаем ссылку на элемент type="file"
    let form = document.querySelector("#frm");
    // обработка события - выбор файла
    document.getElementById('file').onchange = function(e) {
        // -> этот код ...
        node = e.target;
        console.log(node.files[0]); // File
        console.log(node.files[0].name);
        // -> тоже самое, что этод ...
        let data = new FormData (form);
        let file = data.get("file");
        console.log(file); // File
        console.log(file.name);
    }
</script>
```

Метод append для добавления файла

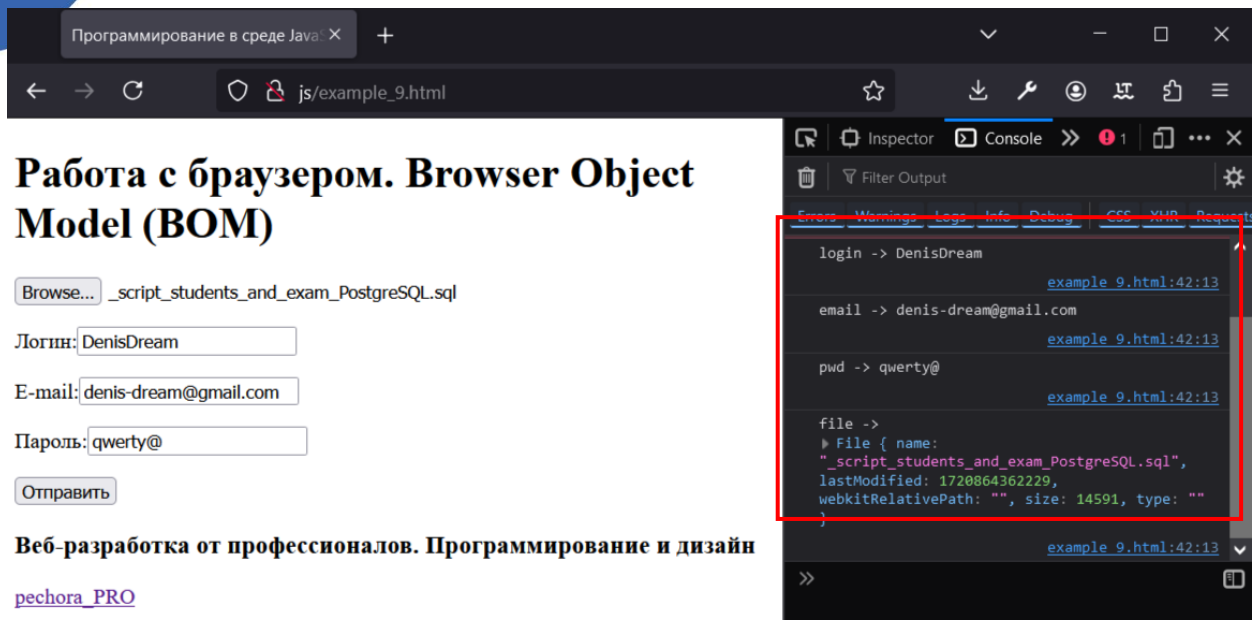
При создании объекта **FormData**, выбранный пользователем файл добавляется к объекту автоматически. Посмотрим, как можно добавить файл, используя метод **append**().

example_9.

```
<!-- форма выбора файла -->
<form id="frm-file">
```

```
    <input id="file" type="file" name="file">
</form><p>
<!-- форма для отправки на сервер -->
<form id="frm-data" onsubmit="return fnSubmit()">
    Логин:<input type="text" name="login" value="DenisDream"><p>
    E-mail:<input type="text" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль:<input type="text" name="pwd" value="qwerty@"><p>
    <input type="submit" value="Отправить">
</form>
<script>
    // обработчик отправки формы
    function fnSubmit() {
        // создаем объект FormData
        let form = document.getElementById('frm-data');
        let data = new FormData(form);
        // получаем ссылку на элемент выбора файла
        let file = document.getElementById('file');
        // добавляем к объекту файл
        data.append('file', file.files[0]);
        // вывод содержимого объекта
        for(let [key, val] of data.entries()){
            console.log(key, '->', val);
        }
        return false;
    }
```

```
</script>
```



При выполнении мультифайловой загрузки первый аргумент стоит объявить **массивом**:

```
// массив в качестве первого аргумента
form_data.append("file[]", file_first.files[0]);
form_data.append("file[]", file_second.files[0]);
```

С помощью метода **append**() можно отправить **динамически сгенерированные бинарные** файлы. Для этого используется синтаксис с тремя аргументами, в качестве третьего аргумента указывается **имя файла**, которое обычно, при <input type="file">, берётся из файловой системы.

Такой способ отправки рассмотрим в теме: **Асинхронный JavaScript**.

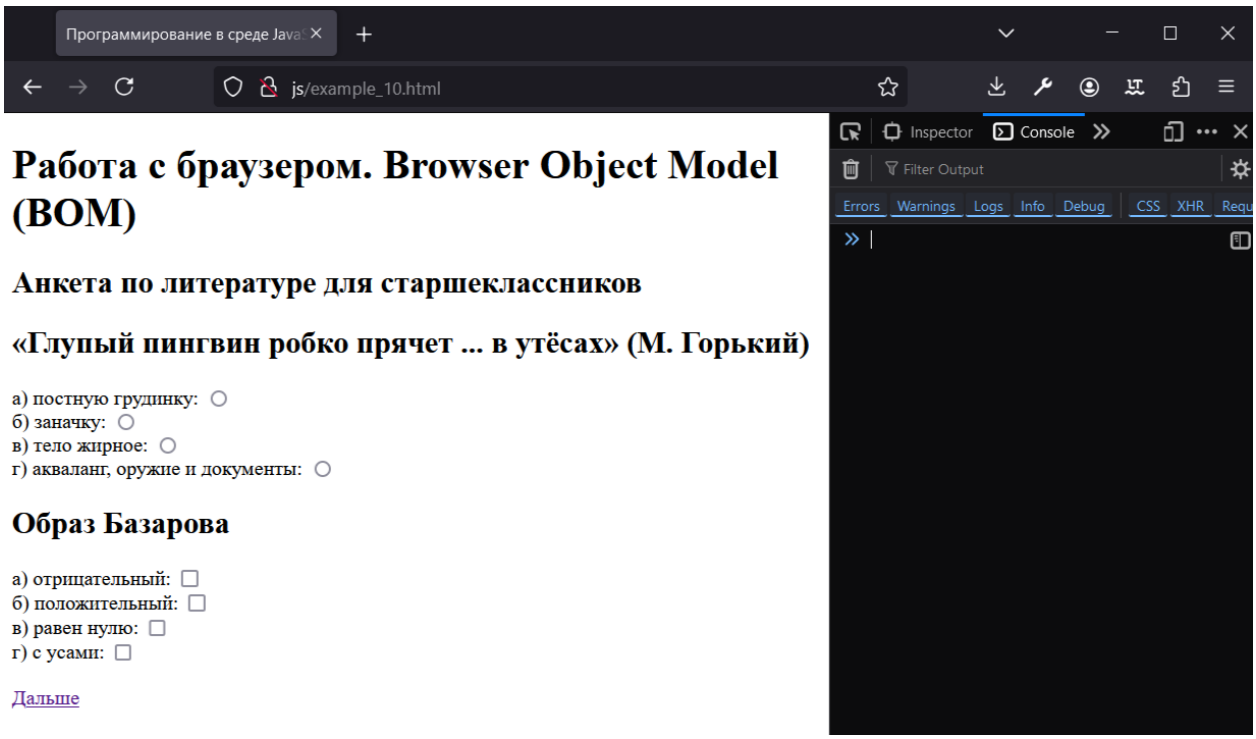
Практика работы с формами

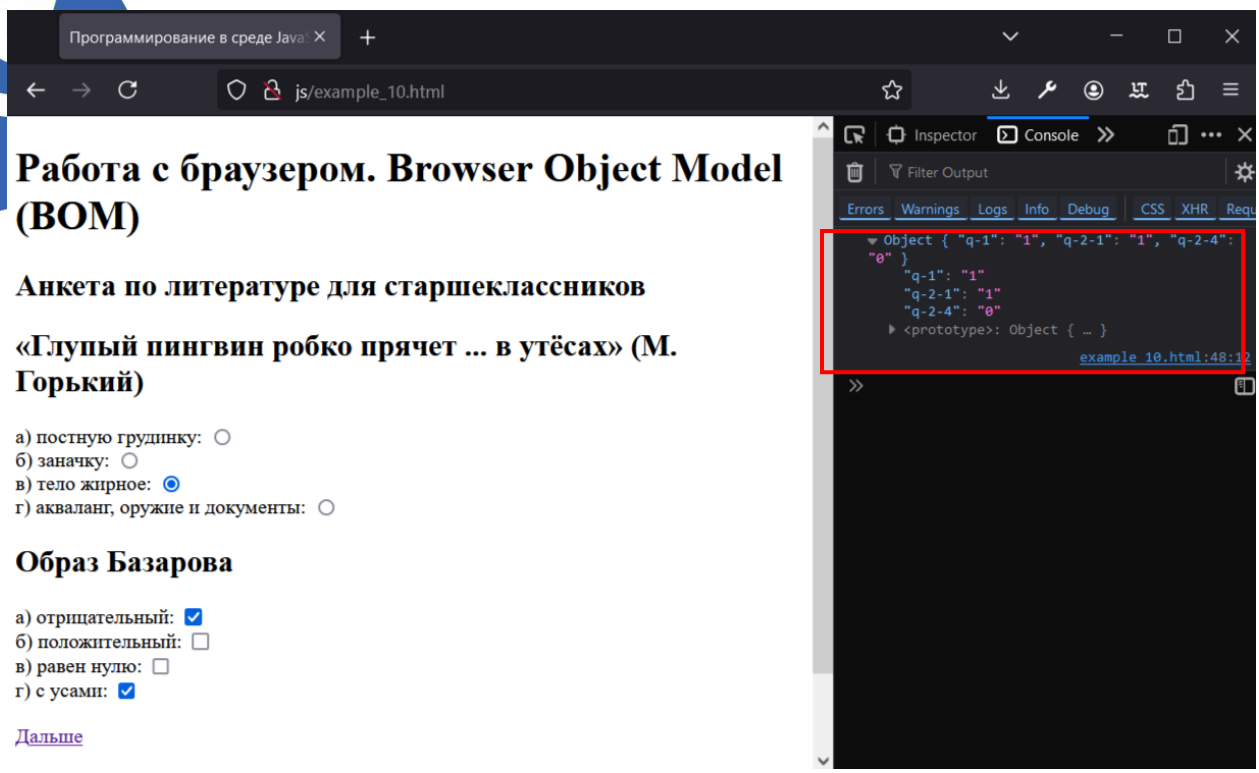
Подводя итоги двум урокам, посвященным работе с элементами формы и объекту **FormData** приведу пример кодинга некоторых часто встречаемых в веб-разработке задач.

Примечание. Форма – один из основных элементов формирования пользовательских данных при организации клиент-серверного приложения.

Легкое обращение с формами сделает разработку асинхронного приложения – увлекательным процессом.

Объект FormData сохраняет только выбранные элементы





example_10.

```
<h2>Анкета по литературе для старшеклассников</h2>

<form id="frm">

    <h2>«Глупый пингвин робко прячет ... в утёсах» (М. Горький)</h2>

        а) постную грудинку: <input type="radio" name="q-1" value="0"><br>

        б) заначку: <input type="radio" name="q-1" value="0"><br>

        в) тело жирное: <input type="radio" name="q-1" value="1"><br>

        г) акваланг, оружие и документы: <input type="radio" name="q-1" value="0"><br>

    <h2>Образ Базарова</h2>

        а) отрицательный: <input type="checkbox" name="q-2-1" value="1"><br>
```

```
        б) положительный: <input type="checkbox" name="q-2-2"
        value="0"><br>
        в) равен нулю: <input type="checkbox" name="q-2-3"
        value="0"><br>
        г) с усами: <input type="checkbox" name="q-2-4"
        value="0"><br>
</form>
<p><a href="#" onclick="fnSubmit(event)">Дальше</a>
<script>
    function fnSubmit(e) {
        e.preventDefault();
        // создаем объект FormData
        // не все поля формы попадут в объект FormData
        let data = new FormData(document.getElementById("frm"));
        // создаем пользовательский объект
        let questions = {}
        // сохраняем в пользовательский объект данные формы
        for(let [key, val] of data) {
            questions[key] = val;
        }
        // вывод в консоль
        console.log(questions)
    }
</script>
```

Примечание. Вернемся к анкете в самостоятельном задании к уроку



Многостраничные формы ;)

Создаем элементы формы на лету

example_11. example_11.html

```
<form id="frm">
    <!-- -->
    <input type="submit" value="Отправить">
</form>
<script>
    // обходим массив в цикле
    // формируем форму для редактирования данных пользователя
    user.forEach( obj => {
        if (obj.login[1] == "DenisDream") {
            inputs = '';
            for(key in obj) {
                inputs += `
                    <span>${obj[key][0]}</span>
                    <input type="text" name="${key}"
                    value="${obj[key][1]}"><p>
                `
            }
        }
    })
    // вставка элементов в форму
    document.getElementById("frm").insertAdjacentHTML("afterbegin
    ", inputs)
```

```
</script>
```

В веб-приложении данные о пользователях, скорее всего, будут храниться в базе данных на стороне сервера (например, MySQL) или в хранилище на стороне клиента (например, IndexDB). Все эти варианты будем разбирать позже.

example_11. user.js

```
let user = [
    {
        login: ['Логин', 'master'],
        email: ['E-mail', 'master@mail.ru'],
        pwd: ['Пароль', '12345']
    },
    // ...
]
```

Простая отправка формы

example_12. example_12.html

```
<form id="frm" action="target.html" method="get">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>E-mail:</span>
    <input type="text" name="email" value="denis-
    dream@gmail.com"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"></span><p>
    <input type="submit" value="Отправить">
```

```
</form>
```

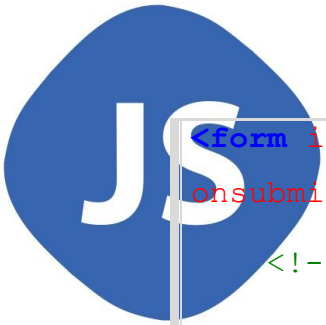
example_12. target.html

```
<script>
    // извлекаем переметры из строки запроса
    let params = new URLSearchParams(location.search);
    // console.log(params);
    // вывод данных формы
    for ([key, val] of params.entries()) {
        document.write (key, ' = ', val, '<br>');
    }
</script>
```

Используем скрытое поле для отправки данных формы

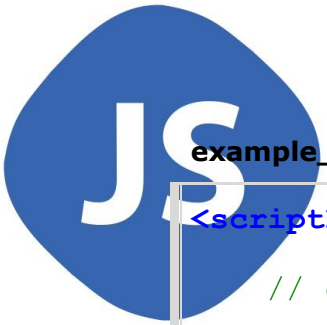
example_13. example_13.html

```
<!-- форма ввода данных -->
<form id="frm-auth">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>E-mail:</span>
    <input type="text" name="email" value="denis-
    dream@gmail.com"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"></span><p>
</form>
<!-- форма отправки данных на сервер -->
```



```
<form id="frm-data" action="target.html" method="GEt"
onsubmit="return fnSubmit()">
    <!--
    скрытое поле, данные этого поля отправятся в файл обработчик
    для наглядности поле не имеет атрибут type='hidden'
    -->
    <input type="text" name="data" value=""><p>
    <input type="submit" value="Отправить">
</form>
<script>
    // обработчик отправки формы
    function fnSubmit() {
        let form = document.getElementById('frm-auth');
        let data = new FormData(form);
        // формирование JSON
        let data_json = JSON.stringify({
            login: data.get("login"),
            email: data.get("email"),
            pwd: data.get("pwd")
        });
        // вставка данных в скрытое поле
        document.getElementById("frm-
        data").elements["data"].value = data_json;
        // console.log(data_json);
        return true;
    }
</script>
```





example_13. target.html

```
<script>
    // создаем из адресной строки параметров объект
    let params = new URLSearchParams(location.search);
    // сохраняем содержимое параметра data
    data_json = params.get("data");
    data = JSON.parse(data_json);
    // вывод данных формы
    document.write ('login = ', data.login, '<br>');
    document.write ('email = ', data.email, '<br>');
    document.write ('pwd = ', data.pwd, '<br>');
</script>
```

Используем ссылку для отправки данных формы

example_14. example_14.html

```
<form id="frm">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>E-mail:</span>
    <input type="text" name="email" value="denis-
    dream@gmail.com"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"></span><p>
</form>
<a id="link" href="target.html" onclick="fnSubmit()">Переход</a>
<script>
```



```
    // обработчик клика по ссылке
    function fnSubmit() {
        let data = new FormData(document.getElementById("frm"));
        let params = new URLSearchParams();
        for ([key, val] of data.entries()) {
            params.append(key, val);
        }
        // добавим к ссылке параметры
        let link = document.getElementById("link");
        let href = link.getAttribute("href");
        let new_href = href + "?" + params.toString();
        link.setAttribute("href", new_href);
    }
</script>
```

example_14. target.html

```
<script>
    // создадим объект из параметров строки запроса
    let params = new URLSearchParams(location.search);
    // console.log(params);
    // вывод данных формы
    for ([key, val] of params.entries()) {
        document.write (key, ' = ', val, '<br>');
    }
</script>
```



Используем объект localStorage

Обратите внимание, если в предыдущих примерах данные передавались в скрипты других файлов, то в случае с объектом **localStorsge**, данные хранятся в хранилище браузера и ждут, когда будут запрошены следующей страницей приложения.

example_15. **example_15.html**

```
<form id="frm">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>E-mail:</span>
    <input type="text" name="email" value="denis-
    dream@gmail.com"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"></span><p>
</form>
<a href="target.html" onclick="fnSubmit()">Отправить данные</a>
<script>
    // обработчик клика по ссылке
    function fnSubmit() {
        // создаем объект FormData
        let form = document.getElementById('frm');
        let data = new FormData(form);
        // преобразуем к массиву и сериализуем данные
        dataJSON = JSON.stringify(Array.from(data));
        // сохраним данные в localStorage
        localStorage.setItem("data", dataJSON);
```

```
    }
</script>
```

example_15. target.html

```
<script>
    // заберем данные из localStorage
    let dataJSON = localStorage.getItem("data");
    // парсим строку в объект
    data = JSON.parse(dataJSON);
    console.log(data);
    // вывод данных формы
    for ([key, val] of data) {
        document.write (key, ' = ', val, '<br>');
    }
</script>
```

Используем свойство oninput

example_16. example_16.html

```
<form id="frm">
    <span>Логин:</span>
    <input type="text" name="login"
    oninput="data.login=this.value"><p>
    <span>E-mail:</span>
    <input type="text" name="email"
    oninput="data.email=this.value"><p>
    <span>Пароль:</span>
```

```
    <input type="text" name="pwd"
    oninput="data.pwd=this.value"></span><p>
</form>
<a id="link" href="#">Отправить данные</a>
<script>
    // инициируем объект сбора пользовательских данных
    let data = {};
    // навешиваем событие на ссылку
    document.getElementById("link").addEventListener("click", e
    => {
        // формируем строку параметров
        search =
        `login=${data.login}&email=${data.email}&pwd=${data.pwd}`
        ;
        // создаем атрибут href для ссылки
        e.target.setAttribute("href", "target.html?" + search);
    });
</script>
```

example_16. target.html

```
<script>
    // создаем объект URLSearchParams
    let params = new URLSearchParams(location.search);
    // вывод данных формы
    document.write ('login = ', params.get('login'), '<br>');
    document.write ('email = ', params.get('email'), '<br>');
    document.write ('pwd = ', params.get('pwd'), '<br>');
```

```
</script>
```

Пример асинхронного обмена данными

Сбор данных формы — это не цель, это промежуточный этап в сценариях обмена данными между приложениями. Немного забегу вперед и приведу несколько примеров, демонстрирующих использование объекта **Formdata** в клиент-серверных приложениях.

> ***Примечание***. Полный разбор асинхронного взаимодействия представлен в теме: **Асинхронный JavaScript**.

> ***Примечание***. В примерах раздела используются файлы **серверных** сценариев.

Сетевой метод **fetch**() легко принимает в качестве значения параметра объект **FormData** (что не обязательно, но очень удобно).

example_17. **exemple_17.html**

```
<form id="frm">
    Логин:<input type="text" name="login" value="DenisDream"><p>
    E-mail:<input type="text" name="email" value="denis-dream@gmail.com"><p>
    Пароль:<input type="text" name="pwd" value="qwerty@"><p>
    <input type="submit" value="Отправить">
</form>
<div id="response"></div>
<script>
```

```
    // асинхронная отправка данных формы
    frm.onsubmit = async () => {
        event.preventDefault();
        let response = await fetch('response.php', {
            method: 'POST',
            body: new FormData(frm)
        });
        let result = await response.json();
        // вывод в консоль ответа сервера
        console.log(result);
    }
</script>
```

Серверный сценарий сериализует принятые данные и отправляет их обратно клиенту.

example_17. **response.php**

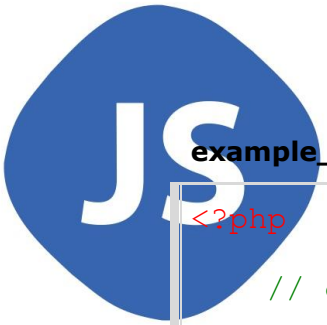
```
<?php
    // ответ сервера
    echo json_encode($_POST, true);
?>
```

В следующем примере выведем принятые от сервера сериализованные данные в документ браузера.

example_18. **exemple_18.html**

```
<form id="frm">
    Логин:<input type="text" name="login" value="DenisDream"><p>
    E-mail:<input type="text" name="email" value="denis-
```

```
        dream@gmail.com"><p>
        Пароль:<input type="text" name="pwd" value="qwerty@"><p>
        <input type="submit" value="Отправить">
</form>
<div id="response"></div>
<script>
    // асинхронная отправка данных формы
    frm.onsubmit = async () => {
        event.preventDefault();
        let response = await fetch('response.php', {
            method: 'POST',
            body: new FormData(frm) // а вот и наш объект
            FormData
        });
        let result = await response.json();
        let msg = `
            <h3>Вы отправили на сервер и получили обратно
            следующие данные:</h3>
            Фамилия:<b> ${result.login} </b><br>
            Имя:<b> ${result.email} </b><br>
            Отчество:<b> ${result.pwd} </b>
        `
        // вставка в документ ответа сервера
        document.getElementById("response").innerHTML = msg;
    }
</script>
```



example_18. **response.php**

```php
<?php
    // ответ сервера
    echo json_encode($_POST, true);
?>
```

В демонстрационном примере **example_19** пример работы с пользовательскими файлами.

example_19. **exemple_19.html**

```html
<form id="frm">
    <input id="file" type="file" name="file"><p>
    <input type="submit" value="Отправить">
</form>
<script>
    // асинхронная отправка данных формы
    frm.onsubmit = async () => {
        event.preventDefault();
        let response = await fetch('response.php', {
            method: 'POST',
            body: new FormData(frm)
        });
        let result = await response.json();
        // вывод в консоль ответа сервера
        console.log(result);
        console.log(result.file.name);
    }
```

```
</script>
```

example_19. **response.php**

```
<?php

    // ответ сервера

    echo json_encode($_FILES, true);

?>
```

Больше примеров и практики в **самостоятельных работах**:

- Добавление названия пользовательского файла к данным формы.
- Создание скрипта для динамического добавления специального элемента выбора файла.
- Создание скрипта для динамического удаления специального элемента выбора файла.
- Вывод в консоль данных динамически формируемой формы пользователя.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.



К **курсу**:

- **Дополнительный практикум.**

